

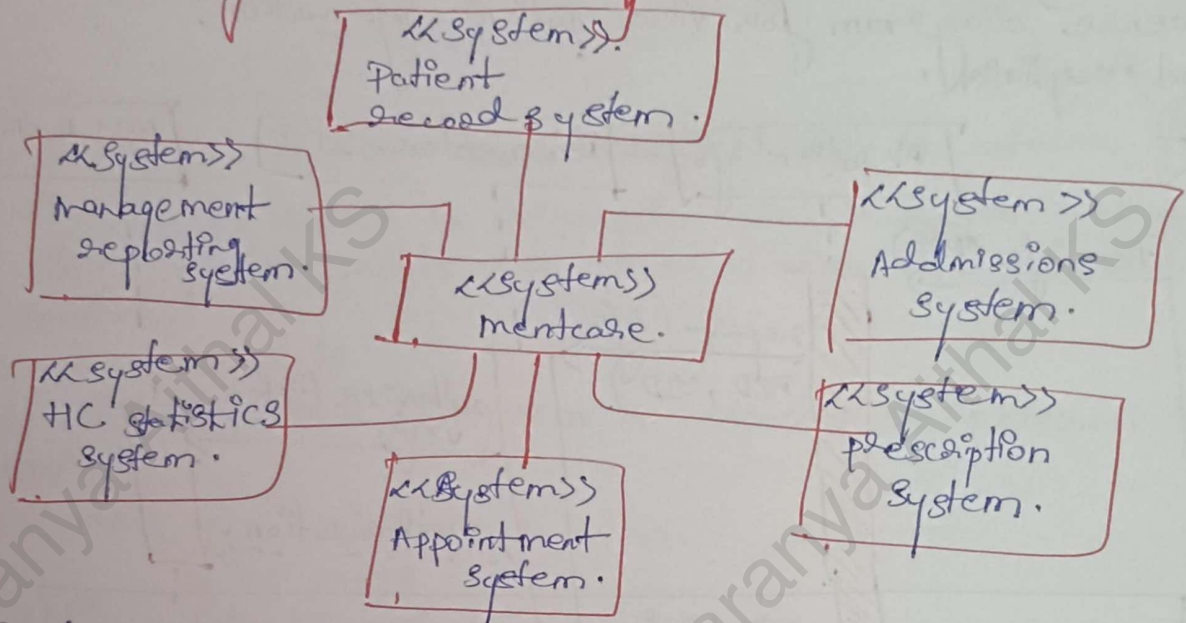
① System modeling.

It is a process of developing abstract models of a system, with each model presenting a different view of that system.

* Context models:

→ These are used to illustrate operational context of a system - they show what lies outside the system boundaries.

* Context model for mentcase-system:



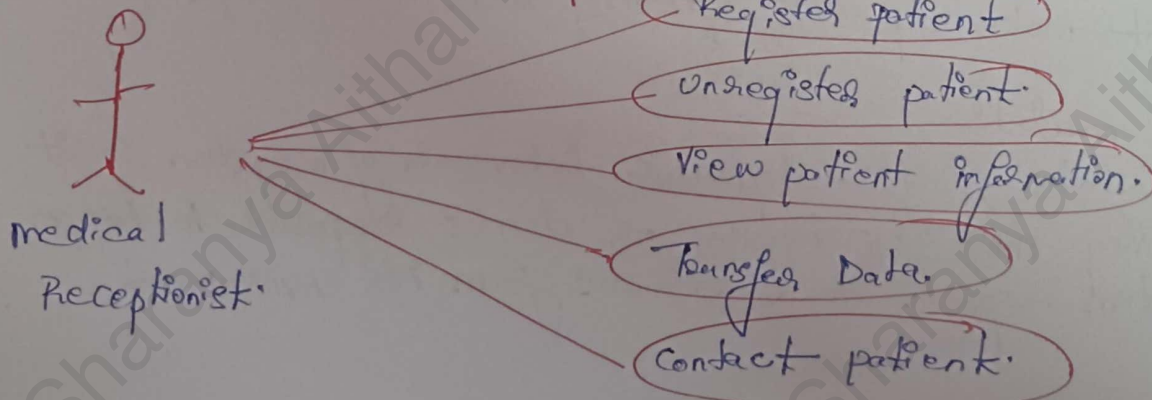
→ They simply show other system in the environment not how the system being developed is used in that environment.

* Interaction models:

It helps us understand if a proposed system structure is likely to deliver the required system performance & dependability.

→ Use case & sequence diagram is used for interaction model.

→ Use case involving Receptionist:



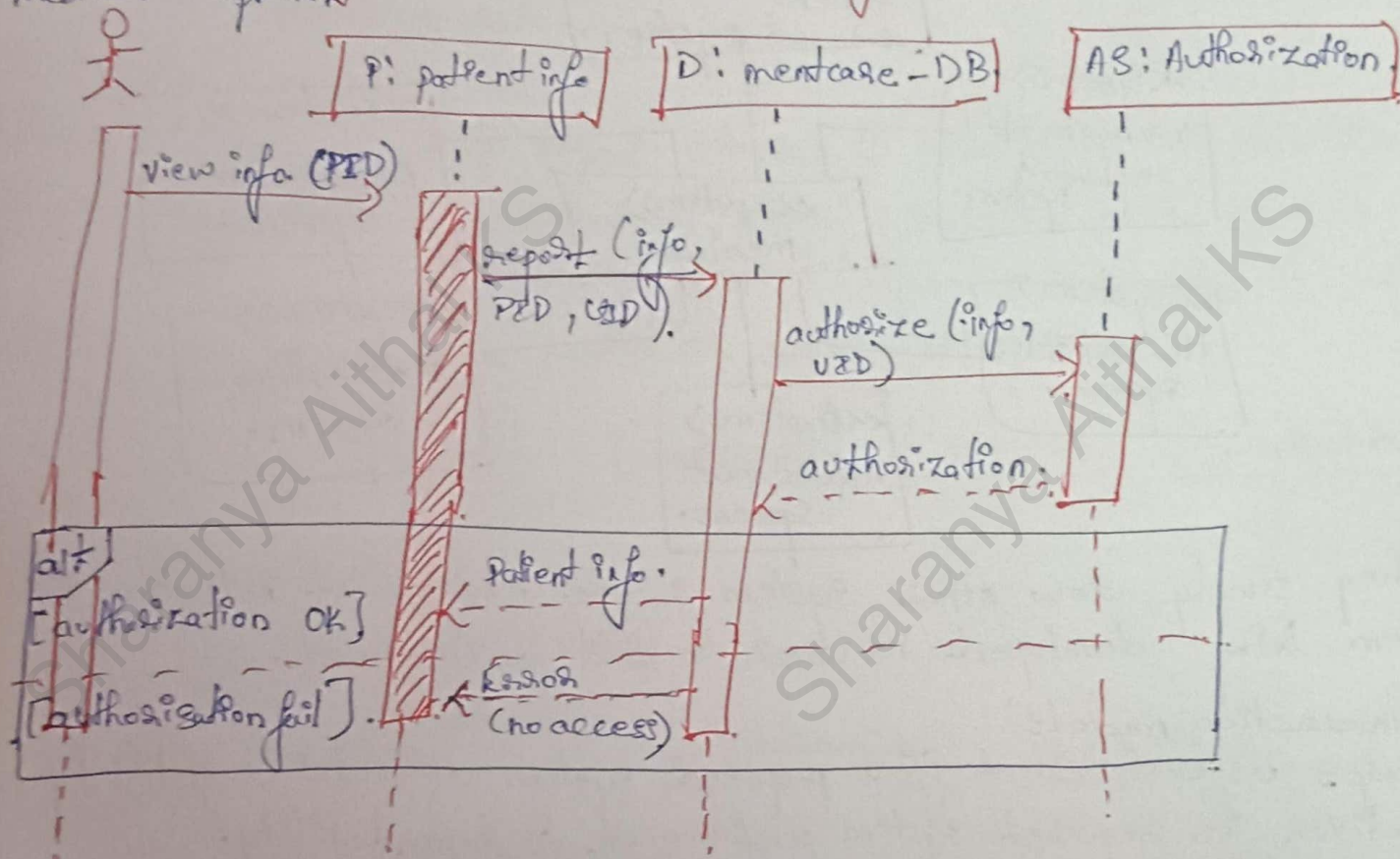
Use case represents discrete task that involves external entities with the system.

→ Sequence diagrams are part of Unified modelling language [UML] & are used to model the interactions between actors & the objects within a system.

→ Sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.

→ Sequence diagram for view patient information!

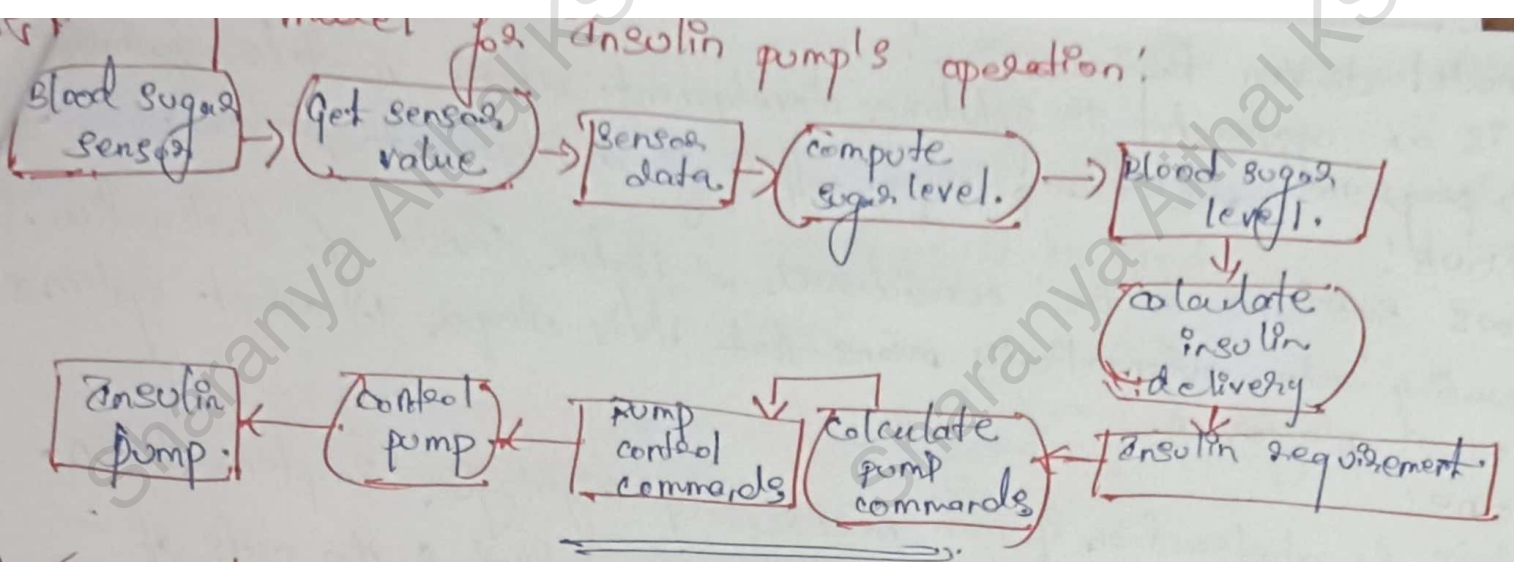
Medical Receptionist.



* **Structural model**: These display the organization of a system in terms of the components that make up that system & their relationships.

* **Behavioral models**:

These are models of the dynamic behaviour of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment.



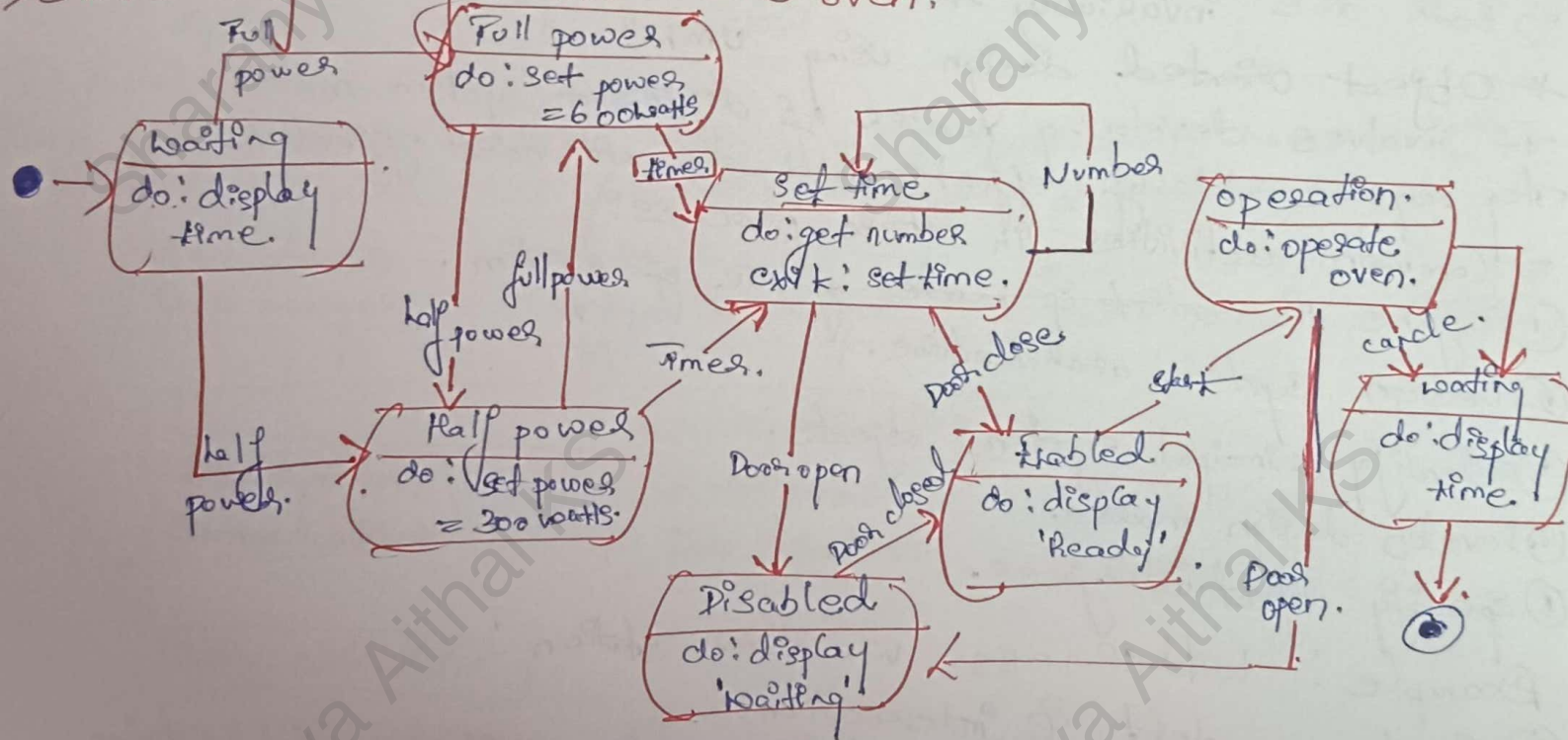
Event driven modeling:

These show how a system ~~represents~~ responds to external & internal events. It is based on assumption that a system has a finite number of states & that events may cause a transition from one state to another.

State machine models:

These model the behaviour of system in response to external & internal events.

State diagram of a microwave oven.



③ Object class identification

* Model driven Engineering:

It is an approach to software development where models rather than programs are the principal outputs of development process.

→ Pros:

Allows systems to be considered at higher levels of abstraction. Generating code automatically means that it is cheaper to adapt systems to new platforms.

→ Cons:

Models for abstraction if not necessarily right for implementation. Savings from generating code may be outweighed by the costs of developing translators for new platforms.

② Design Implementation.

This is a stage in the software engineering process at which an executable software system is developed.

→ Both are invariably interleaved.

* Object oriented design using UML:

It involves developing number of different system models. They require lot of efforts and are not cost effective.

→ Common activities in these process:

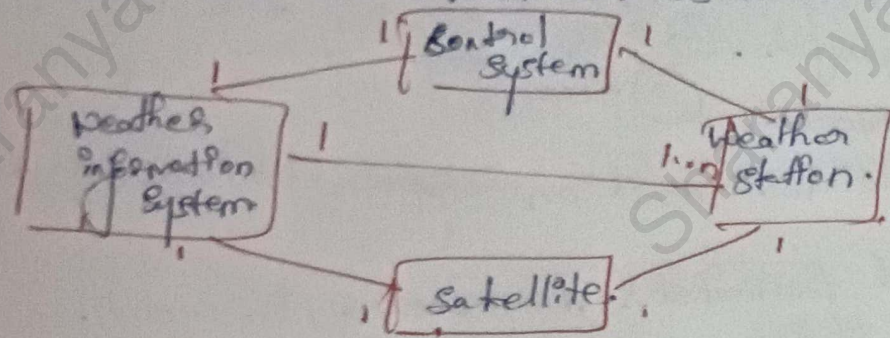
- ① Define the context & modes of use of system.
- ② Design system architecture.
- ③ Identify principal system objects.
- ④ Develop design model.
- ⑤ Specify object interfaces.

Example: Wilderness weather station.

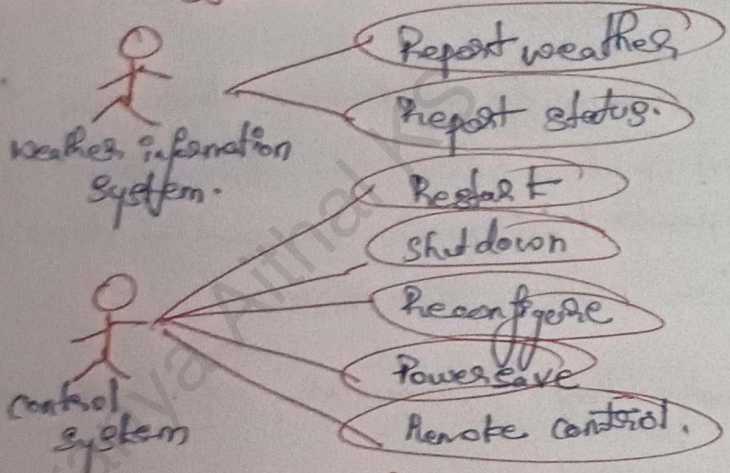
① System context & interaction:

Understanding the relationships between the software that is being designed & its external environment is essential for deciding how to provide required system functionality & how to structure the system to communicate with its environment.

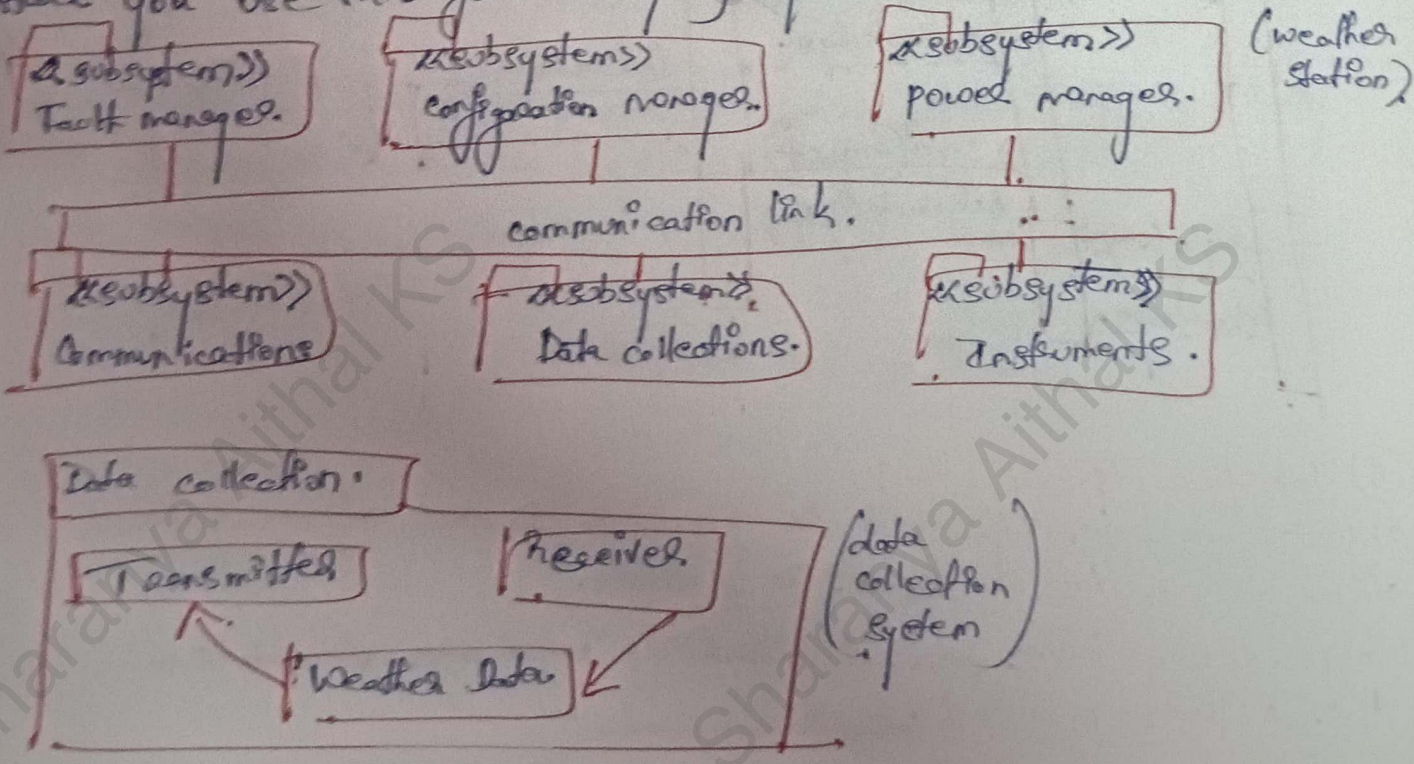
Context model is a structural model that demonstrates other systems in environment of system being developed.
 Interaction model is a dynamic model that shows how the system interacts with its environment as it is used.



① Weather station use cases: (weather report)



② Architectural Design:
 Once interactions between system & its environment have been understood you use this for designing system architecture.



③ Object class identification:

It relies on skill, experience & domain knowledge of system designers.
→ It is an interactive process.

→ Approach to identification:

- ① Grammatical approach. ④ Base identification on tangible in the application domain.
② Behavioural approach. ③ Scenario based analysis.

→ For weather station: based on tangible hardware & data in the system.

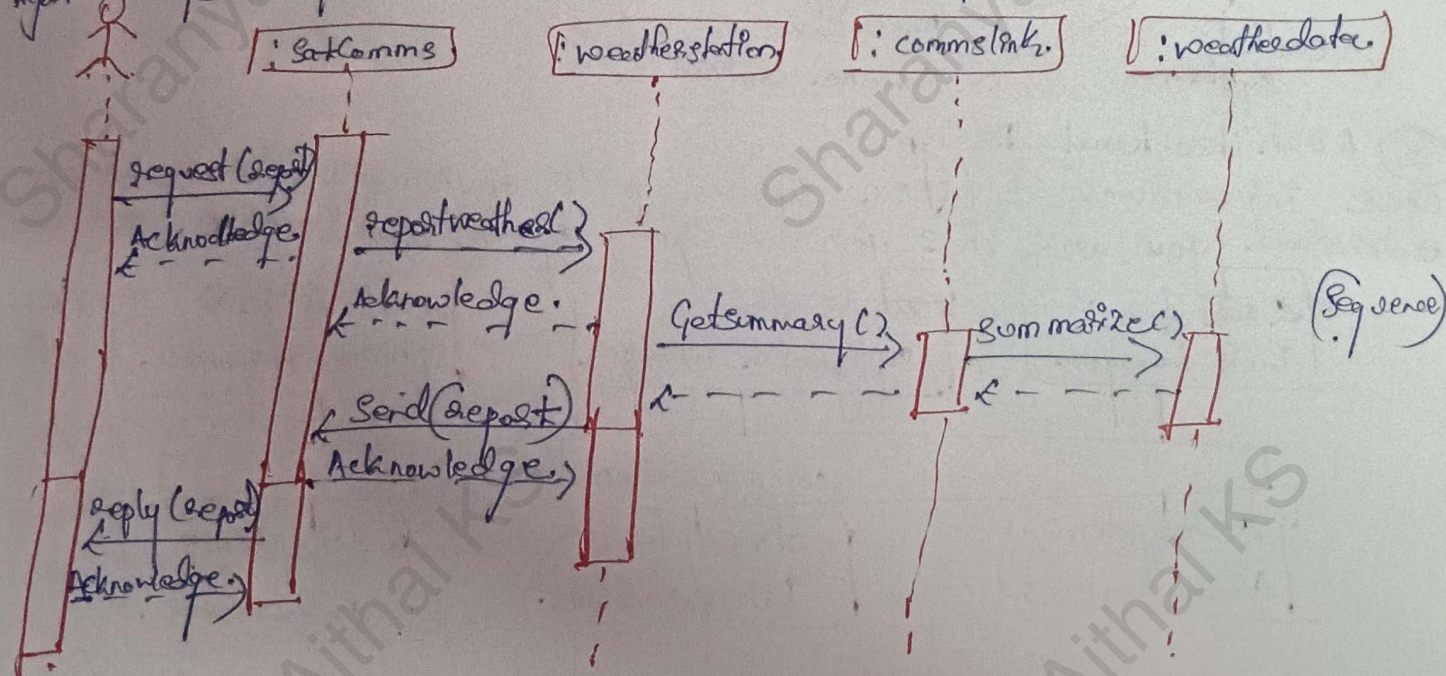
- General thermometers, Anemometer, Barometer.
- Weather station.
- Weather data.

④ Design models:

These show object & object classes & relationships between them.

→ 2 Types — Structural model: Describes static structure.
— Design model: Dynamic interaction b/w objects.

→ This may be sequence model or state diagram.

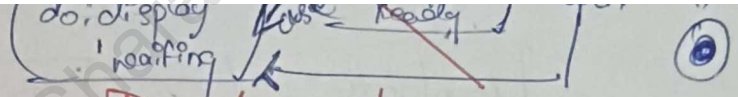


⑤ Interface Specification:

object interfaces have to be specified so that objects & other components can be designed in parallel.

→ objects may have several interfaces which are viewpoints on the methods provided.

③ Agile Software Development



* Agile Principles:

① Customer Involvement:

- Customer should be closely involved throughout the process.
- Role: provide & prioritize new system requirements & to evaluate the iterations of the system.

② Incremental Delivery:

- The software is developed in increments with the customer specifying the requirements to be included in each increment.

③ People not process:

- The skills of development team should be recognized & exploited
- Team members should be left to develop their own ways of working without prescriptive processes.

④ Embrace change:

- Expect the system requirements to change & so design the system to accommodate these changes.

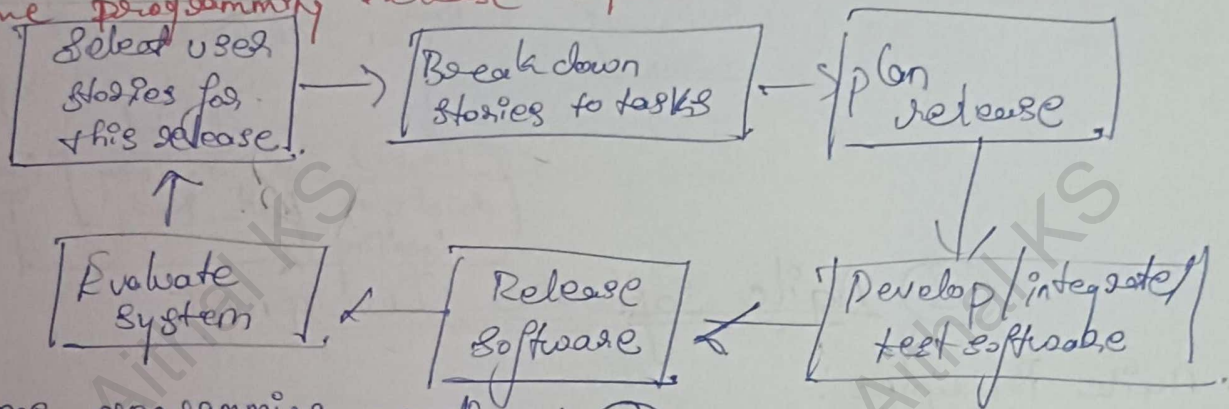
⑤ Maintain Simplicity:

- Focus on simplicity in both the software being developed & in the development process, wherever possible actively work to eliminate complexity from the system.

* Extreme programming:

- popular form of Agile.
- It takes an extreme approach to iterative development:
 - New versions may be built several times per day.
 - Increments are delivered to customers every 2 weeks.
 - All tests must be run for every build & the build is only accepted if tests run successfully.
- Agile principles (All 5.)

⇒ Extreme programming release cycle:



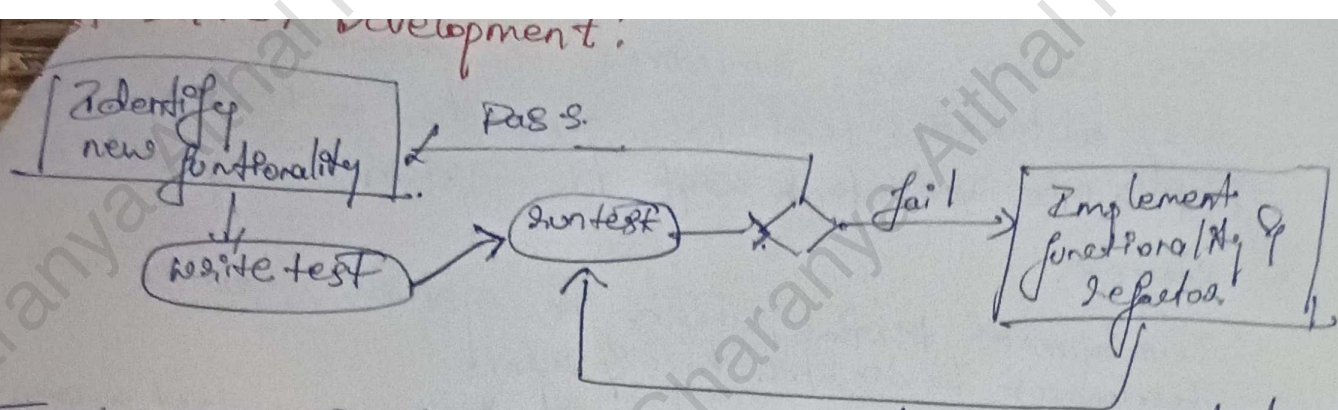
→ Extreme programming practices: (a).

- ① Incremental planning: take opinion from ppl.
- ② Small releases: Release first version.
- ③ Simple design: Design a project in simple way.
- ④ Test first development: Testing project each level.
- ⑤ Refactoring: Improvement & removing error.

→ Extreme programming practices: (b).

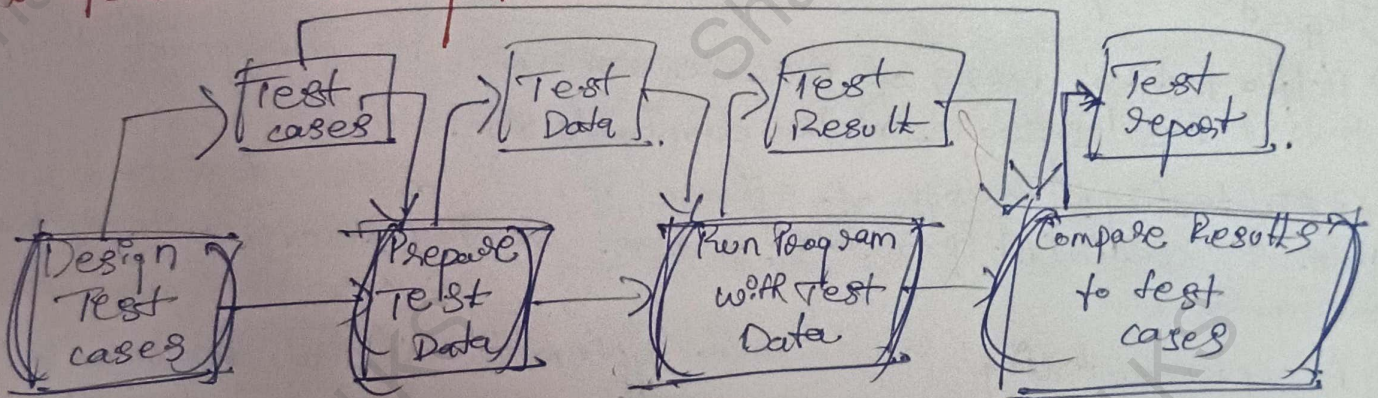
- ① Pair programming: programmers working in pairs, developing code together.
- ② collective ownership: Anyone can change project at any stage.
- ③ continuous integration: After completion unit tests on system is done.
- ④ Sustainable pace!
- ⑤ on site customer: Customer is member of development team & responsible for being System Requirements.

④ Testing



- Test Driven Development is an approach to program development in which you inter-leave testing & code development.
- Tests are written before code & passing the tests is the critical driver of development.
- You develop code incrementally, along with a test for that increment. You don't move on the next increment until the code that you have developed passes its test.
- TDD was introduced as part of agile methods such as Extreme programming. However it can also be used in plan driven development processes.

General Testing model:



- Test cases are specifications of the inputs to the test & expected output from the system, plus a statement of what is being tested.
- Test data are the inputs that have been devised to test a system.

→ Test data can sometimes be generated automatically but automatic test case generation is challenging as people who understand what the system is supposed to do.

→ Stages:

→ Development testing, where the system is tested during development to discover bugs & defects.

→ Release testing: where a separate testing team test a complete version of system before it is released to users.

→ User testing: where users or potential users of a system test the system in their own environment.

* Benefits of Test-Driven Development:

- ① Code coverage
- ② Regression testing.
- ③ Simplified debugging.
- ④ System documentation.

~~A~~ User testing: stage in which ~~testing~~ users provide input & advice on system testing.

→ Types:

① Alpha testing: users of software work with the development team to test the software at the developer's site.

② Beta testing: release of software is made available to users to allow them to experiment & to raise problems that they discover with system developers.

③ Acceptance testing: users test system to decide whether or not it is ready to be accepted from the system developers & deployed in the ~~customer~~ customer environment.

